



TESTE DE SOFTWARE
Implementação de Padrões de Teste

Larissa Pedrosa Silva
801161

Belo Horizonte
2025

1. Padrões de Criação de Dados (Builders)

a. Porque CarrinhoBuilder em vez de CarrinhoMother?

O CarrinhoBuilder foi escolhido em vez do CarrinhoMother devido à complexidade e variabilidade do objeto Carrinho. Enquanto o *Object Mother* é indicado para entidades simples e fixas, o *Data Builder* é mais flexível para lidar com objetos complexos que mudam entre os testes.

- i. **Variabilidade de Configuração:** Um carrinho pode ter diferentes usuários, múltiplos itens, estar vazio, ou ter valores específicos. Criar um método Mother para cada combinação levaria a uma explosão de métodos.
- ii. **Legibilidade:** O Builder permite que o teste seja auto explicativo, mostrando apenas o que é relevante para aquele cenário específico. Isso resolve o Test Smell de "Setup Obscuro".
- iii. **Manutenibilidade:** Se a estrutura do **Carrinho** mudar, apenas o Builder precisa ser atualizado, não múltiplos métodos Mother.

b. Antes vs. Depois

i. Antes

```
const usuario = new User(2, 'Maria Santos', 'premium@email.com', 'PREMIUM');
const item1 = new Item('Produto 1', 100);
const item2 = new Item('Produto 2', 100);
const itens = [item1, item2];
const carrinho = new Carrinho(usuario, itens);
```

Os principais problemas identificados foram o excesso de linhas de código necessárias para criar um objeto simples, a presença de detalhes irrelevantes como nomes de produtos e IDs, que acabam poluindo o teste, a dificuldade em identificar quais informações são realmente essenciais para o cenário testado e a repetição de código em diversos testes, tornando a manutenção mais trabalhosa.

ii. Depois

```
1 const carrinho = new CarrinhoBuilder()
2   .comUser(usuarioPremium)
3   .comValorTotal(200)
4   .build();
```

Os benefícios de usar esta abordagem são a legibilidade, pois o teste mostra de forma clara apenas o que é relevante para o cenário; a concisão, com menos linhas de código; a flexibilidade, que permite customizar facilmente apenas o necessário; e a manutenibilidade, já que qualquer mudança no Builder é refletida automaticamente em todos os testes.

c. Como o Builder melhora a legibilidade e manutenção

O uso do Builder melhora a legibilidade e a manutenção dos testes. Em termos de legibilidade, o código passa a ser mais intuitivo e fácil de entender. Além disso, o Builder remove ruídos visuais, eliminando detalhes irrelevantes e facilitando a compreensão do cenário testado.

Quanto à manutenibilidade, o Builder centraliza as mudanças: qualquer alteração na estrutura do objeto Carrinho é feita apenas nele, sem impactar diretamente os testes. Também promove a reutilização, já que o mesmo Builder pode ser utilizado em diferentes testes, e favorece a evolução do código, permitindo adicionar novos métodos de

customização sem quebrar testes existentes.

Por fim, o Builder ajuda a prevenir Test Smells, eliminando o “Setup Obscuro” ao tornar o preparo do teste mais claro e focado, reduzindo a duplicação de código e facilitando a criação de testes mais específicos e isolados.

2. Padrões de Test Doubles (Mocks vs. Stubs)

a. Análise do Teste "Sucesso Premium"

No teste “quando um cliente Premium finaliza a compra”, foram utilizados diferentes tipos de Test Doubles:

```
1 // Stub: GatewayPagamento
2 const gatewayStub = {
3   cobrar: jest.fn().mockResolvedValue({ success: true })
4 };
5
6 // Stub: PedidoRepository
7 const pedidoSalvo = new Pedido(123, carrinho, 180, 'PROCESSADO');
8 const repositoryStub = {
9   salvar: jest.fn().mockResolvedValue(pedidoSalvo)
10 };
11
12 // Mock: EmailService
13 const emailMock = {
14   enviarEmail: jest.fn().mockResolvedValue(undefined)
15 };
```

b. Identificação: Stub vs. Mock

O **GatewayPagamento** foi implementado principalmente como um Stub, pois sua função principal é fornecer uma resposta pré-definida neste caso, { success: true } para controlar o fluxo do teste. Nesse contexto, o foco não está em verificar o comportamento do método, mas sim o estado, ou seja, o valor passado para o processo de cobrança. O objetivo é garantir que o desconto de 10% seja aplicado corretamente (de R\$ 200 para R\$ 180). A verificação é feita por meio da instrução: **expect(gatewayStub.cobrar).toHaveBeenCalledWith(180, cartaoCredito)**, que confirma o valor calculado e não apenas se o método foi chamado, validando assim o resultado esperado do teste.

O **EmailService** foi implementado como um Mock, pois sua função principal é verificar o comportamento do sistema, isto é, as interações realizadas e não apenas fornecer dados. Nesse caso, o teste valida se o serviço de e-mail foi chamado, quantas vezes isso ocorreu e com quais argumentos, garantindo que o e-mail seja enviado corretamente após um pagamento bem-sucedido. As verificações realizadas:

expect(emailMock.enviarEmail).toHaveBeenCalledTimes(1) e:
expect(emailMock.enviarEmail).toHaveBeenCalledWith('premium@email.com', 'Seu Pedido foi Aprovado!', 'Pedido 123 no valor de R\$180'), confirmam o comportamento esperado e a interação adequada com o serviço externo.

Já o **PedidoRepository** foi implementado como um Stub, cuja função principal é fornecer uma resposta pré-definida neste caso, um pedido salvo com um ID para simular a persistência sem depender de um banco de dados real. O teste, portanto, verifica o estado, garantindo que o pedido tenha sido retornado corretamente dentro do fluxo do sistema.

3. Conclusão

O uso de padrões de teste, como *Builders* e *Test Doubles*, ajuda a evitar problemas comuns e torna a suíte de testes mais fácil de manter. Antes, o setup dos testes era manual e complicado, mas com os *Builders* ele ficou mais claro e direto, deixando os testes mais simples de entender. Os *Test Doubles* também ajudam a deixar os testes mais estáveis, pois isolam as dependências externas e evitam que mudanças no sistema afetem tudo. Além disso, os *Builders* centralizam a criação dos objetos, evitando repetição de código e facilitando a manutenção.

Esses padrões também tornam os testes mais legíveis, já que eles funcionam quase como uma documentação viva do sistema, e mais confiáveis, por serem rápidos, estáveis e verificarem exatamente o que deve ser testado. Por fim, a suíte de testes se torna escalável, pois fica mais fácil adicionar novos testes e evoluir os existentes sem quebrar o que já funciona.